

# Lec28

# Express Router

Imagine you're building a website.

It has

- Home Page
- Login Page
- Registration Page
- Student Page
- Teacher Page
- Contact Page
- About Page

If we write everything inside one file (`server.js`), it becomes very long.

# Example

```
app.get("/", ...);
```

```
app.get("/login", ...);
```

```
app.post("/login", ...);
```

```
app.get("/register", ...);
```

```
app.post("/register", ...);
```

```
app.get("/students", ...);
```

```
app.post("/students", ...);
```

```
app.put("/students/:id", ...);
```

```
app.delete("/students/:id", ...);
```

# Solution

Express Router allows us to split our routes into different files.

Think of Router like different departments in a college.

Express App

```
|  
|---- User Routes  
|  
|---- Student Routes  
|  
|---- Teacher Routes  
|  
|---- Product Routes
```

# Router Architecture

app.js

routes/

user.js

student.js

teacher.js

# Creating Router

## **user.js**

```
const express = require("express");
```

```
const router = express.Router();
```

```
router.get("/", (req,res)=>{  
  res.send("User Home");  
});
```

```
module.exports = router;
```

# app.js

```
const express = require("express");
```

```
const app = express();
```

```
const userRoutes = require("./routes/user");
```

```
app.use("/users", userRoutes);
```

```
app.listen(3000);
```

# Multiple Routes

```
router.get("/", (req,res)=>{  
  res.send("All Users");  
});
```

```
router.get("/profile", (req,res)=>{  
  res.send("User Profile");  
});
```

```
router.get("/settings", (req,res)=>{  
  res.send("Settings");  
});
```

# Why Routers?

Easy to understand

Easy to maintain

Better organization

Reusable

Large applications become manageable

# Controllers

A **Controller** is a JavaScript file (or module) that contains the **business logic** of an application.

Its main job is to:

- Receive the request from the client.
- Process the request.
- Interact with the database or model.
- Prepare the response.
- Send the response back to the client.

# Why Do We Need Controllers?

Imagine you have a project with 100 routes.

Without controllers:

```
app.get("/students", (req, res) => {  
  
});  
  
app.post("/students", (req, res) => {  
  
});  
  
app.put("/students/:id", (req, res) => {  
  
});  
  
app.post("/teachers", (req, res) => {  
  
});
```

Everything is inside one file.

Problems:

Very difficult to read

Hard to debug

Hard to maintain

Difficult to work in a team

Code duplication

# Solution

Move all business logic into controllers.

Now,

Route



Controller



Response

Much cleaner.

# Express MVC Architecture

MVC stands for

Model

View

Controller

In REST APIs we usually use

Model

Controller

Routes

because APIs generally return JSON instead of HTML views.

# MVC Diagram

Client



Routes



Controller



Model



Database



Controller



Response

# Request Flow

## Step 1

User sends request

GET /students



## Step 2

Express receives request



## Step 3

Router checks URL

#### **Step 4**

Controller is called



#### **Step 5**

Controller processes request



#### **Step 6**

Database returns data



#### **Step 7**

Controller sends response



#### **Step 8**

User receives JSON

# Creating Controllers

Project Structure

student-api/

controllers/  
  studentController.js

routes/  
  studentRoutes.js

app.js

# studentController.js

```
exports.getStudents = (req, res) => {  
  
  res.send("All Students");  
  
};
```

This is our first controller.

## studentRoutes.js

```
const express = require("express");
```

```
const router = express.Router();
```

```
const studentController = require("../controllers/studentController");
```

```
router.get("/", studentController.getStudents);
```

```
module.exports = router;
```

Notice

Instead of

```
router.get("/", (req, res) => {  
  
});
```

We simply call

```
studentController.getStudents
```

# app.js

```
const express = require("express");
```

```
const app = express();
```

```
const studentRoutes = require("./routes/studentRoutes");
```

```
app.use("/students", studentRoutes);
```

```
app.listen(3000);
```

# Multiple Controllers

Instead of one function,

we create many.

```
exports.getStudents = (req, res) => {  
  
};  
  
exports.addStudent = (req, res) => {  
  
};  
  
exports.updateStudent = (req, res) => {  
  
};  
  
exports.deleteStudent = (req, res) => {  
  
};
```

Each controller performs one task.

# Connecting Routes with Controllers

```
const express = require("express");  
  
const router = express.Router();  
  
const studentController = require("../controllers/studentController");  
  
router.get("/", studentController.getStudents);  
  
router.get("/:id", studentController.getStudent);  
  
router.post("/", studentController.addStudent);  
  
router.put("/:id", studentController.updateStudent);  
  
router.delete("/:id", studentController.deleteStudent);  
  
module.exports = router;
```

This keeps the route file very small and easy to read.

# CRUD Operations

CRUD is the heart of backend development.

CRUD stands for

C → Create

R → Read

U → Update

D → Delete

Every website performs these four operations.

# Example

Instagram

Create

Post Photo

Read

View Feed

Update

Edit Caption

Delete

Delete Post

# CRUD with HTTP Methods

| CRUD   | HTTP Method | Purpose            |
|--------|-------------|--------------------|
| Create | POST        | Insert data        |
| Read   | GET         | Retrieve data      |
| Update | PUT         | Update entire data |
| Delete | DELETE      | Remove data        |

# Example Student Object

```
{  
  id:1,  
  name:"Rahul",  
  age:20  
}
```

# CREATE

```
app.post("/students",(req,res)=>{  
  
  res.send("Student Added");  
  
});
```

Request

POST /students

# READ

```
app.get("/students",(req,res)=>{  
  res.send("All Students");  
});
```

# UPDATE

```
app.put("/students/:id",(req,res)=>{  
  res.send("Student Updated");  
});
```

# DELETE

```
app.delete("/students/:id",(req,res)=>{  
  
    res.send("Student Deleted");  
  
});
```

# Complete CRUD Example

```
const express = require("express");
```

```
const app = express();
```

```
app.use(express.json());
```

```
let students = [];
```

```
app.get("/students",(req,res)=>{  
  
  res.json(students);  
  
});  
  
app.post("/students",(req,res)=>{  
  
  students.push(req.body);  
  
  res.send("Student Added");  
  
});
```

```
app.put("/students/:id",(req,res)=>{  
  
  res.send("Student Updated");  
  
});  
  
app.delete("/students/:id",(req,res)=>{  
  
  res.send("Student Deleted");  
  
});  
  
app.listen(3000);
```

# CRUD Controllers

Suppose

```
let students = [  
  {  
    id:1,  
    name:"Rahul",  
    age:20  
  }  
];
```

# GET Controller

```
exports.getStudents = (req, res) => {
```

```
  res.json(students);
```

```
};
```

# GET by ID Controller

```
exports.getStudent = (req, res) => {  
  
  const id = parseInt(req.params.id);  
  
  const student = students.find(s => s.id === id);  
  
  if (!student) {  
  
    return res.status(404).json({  
      message: "Student Not Found"  
    });  
  
  }  
  
  res.json(student);  
  
};
```

# POST Controller

```
exports.addStudent = (req, res) => {  
  
  const student = {  
  
    id: students.length + 1,  
  
    name: req.body.name,  
  
    age: req.body.age  
  
  };  
  
  students.push(student);  
  
  res.status(201).json({  
  
    message: "Student Added",  
  
    student  
  
  });  
  
};
```

# PUT Controller

```
exports.updateStudent = (req, res) => {  
  
  const id = parseInt(req.params.id);  
  
  const student = students.find(s => s.id === id);  
  
  if (!student) {  
  
    return res.status(404).json({  
      message: "Student Not Found"  
    });  
  
  }  
  
  student.name = req.body.name;  
  
  student.age = req.body.age;  
  
  res.json({  
  
    message: "Student Updated",  
  
    student  
  
  });  
  
};
```

# DELETE Controller

```
exports.deleteStudent = (req, res) => {  
  
  const id = parseInt(req.params.id);  
  
  const index = students.findIndex(s => s.id === id);  
  
  if (index === -1) {  
  
    return res.status(404).json({  
  
      message: "Student Not Found"  
  
    });  
  
  }  
  
  students.splice(index, 1);  
  
  res.json({  
  
    message: "Student Deleted"  
  
  });  
  
};
```

# Request Handling

Every time a browser sends a request



Express receives it



Processes it



Returns a response

This process is called

Request Handling

# Request Life Cycle

Browser



Request



Express



Middleware



Route



Database



Response

# Request Object

Express provides

`req`

It contains all information sent by the client.

Example

```
app.get("/",(req,res)=>{  
  
});
```

Here

req

contains

- URL
- Headers
- Query
- Body
- Parameters
- Cookies

etc.

# Response Object

`res`

is used to send data back.

Example

```
res.send("Hello");
```

or

```
res.json(data);
```

# req.params

Used for URL parameters.

Example

/students/5

Route

```
app.get("/students/:id",(req,res)=>{  
    res.send(req.params.id);  
});
```

Output

5

# req.query

Example URL

/search?name=Rahul

Route

```
app.get("/search",(req,res)=>{  
    res.send(req.query.name);  
});
```

Output

Rahul

# req.body

Data sent by POST request.

Example

```
{  
  "name": "Rahul"  
}
```

Need middleware

```
app.use(express.json());
```

Route

```
app.post("/student", (req, res) => {  
  
  console.log(req.body);  
  
});
```

## req.headers

```
app.get("/",(req,res)=>{
```

```
  console.log(req.headers);
```

```
});
```

Shows browser information.

### **res.send()**

```
res.send("Hello");
```

### **res.json()**

```
res.json({
```

```
  name:"Rahul",
```

```
  age:20
```

```
});
```

### **res.status()**

```
res.status(404).send("Not Found");
```

# API Structure

API means

Application Programming Interface

An API allows two applications to communicate.

# Rest API Structure

A good API follows a proper structure.

Example

GET /students

GET /students/5

POST /students

PUT /students/5

DELETE /students/5

# API Folder Structure

Project

controllers/

models/

routes/

middlewares/

app.js

package.json

# Example API Response

```
{  
  "success":true,  
  "message":"Student Found",  
  "data":{  
    "id":1,  
    "name":"Rahul"  
  }  
}
```

| Code | Meaning               | Description                              |
|------|-----------------------|------------------------------------------|
| 200  | OK                    | Request successful                       |
| 201  | Created               | Resource created successfully            |
| 204  | No Content            | Request successful but nothing to return |
| 400  | Bad Request           | Invalid request from client              |
| 401  | Unauthorized          | Authentication required                  |
| 403  | Forbidden             | User does not have permission            |
| 404  | Not Found             | Resource not found                       |
| 409  | Conflict              | Duplicate or conflicting resource        |
| 500  | Internal Server Error | Unexpected server error                  |